1 Real-time SSR corrections

The real-time engine consumes network corrections in *State Space Representation* (SSR): per-satellite, per-signal refinements to the broadcast orbit, clock and hardware biases. They arrive as RTCM 3.x messages (legacy 1057–1068 and 1240–1270) or in the IGS-SSR `4076` repackaging [1, 2]. Each SSR quantity refines one term already present in the undifferenced code equation and the undifferenced phase equation of the Foundations document, and none introduces a new state. SSR is what distinguishes the precise real-time mode from broadcast-only operation: with SSR applied, $c\,\mathrm{d}t^i$ and $\boldsymbol{r}^i$ are centimetre-grade, and the satellite biases B_s^i, b_s^i are *known* instead of absorbed into the ambiguity. This document describes the user-side application of the corrections. The same store receives a third-party IGS-RTS stream and the products of the engine's own network estimator (the Network mode document), so the application mathematics below does not depend on their source.

Decoded records are cached per satellite in `products/SsrCorrectionStore`: a process-wide singleton for the live stream, or a caller-owned instance for the closed-loop user of the network (Section 1.9). The orbit is applied in `model/ModelGeometry`, the clock in `corrections/GnssCorrections` and the biases in `measurement/ObservationEquation`. Each queries the store and falls back to broadcast-only when it holds nothing usable. In post-processing the store is empty, so every equation below reduces to the broadcast-only observation model. The streaming and decoding layer (RTCM frame parsing, the per-satellite mutex-guarded cache, the real-time push loop) is described in the Real-time engine document. This document covers only the store-driven correction mathematics, which does not depend on how the records arrived. There is no `enable_ssr` flag: SSR is applied whenever the store holds a usable record.

1.1 Orbit correction: refines ρ_r^i

The orbit message carries a delta in the satellite's radial, along-track and cross-track (RAC) frame, referenced to the broadcast ephemeris of issue IODE. The frame is built from the broadcast position $\boldsymbol{r}_{\mathrm{brdc}}^i$ and velocity $\dot{\boldsymbol{r}}_{\mathrm{brdc}}^i$ (`products/SsrCorrectionStore.cpp`):

$$\hat{\boldsymbol{e}}_a = \frac{\dot{\boldsymbol{r}}_{\mathrm{brdc}}^i}{\|\dot{\boldsymbol{r}}_{\mathrm{brdc}}^i\|}, \quad \hat{\boldsymbol{e}}_c = \frac{\boldsymbol{r}_{\mathrm{brdc}}^i \times \dot{\boldsymbol{r}}_{\mathrm{brdc}}^i}{\|\boldsymbol{r}_{\mathrm{brdc}}^i \times \dot{\boldsymbol{r}}_{\mathrm{brdc}}^i\|}, \quad \hat{\boldsymbol{e}}_r = \hat{\boldsymbol{e}}_a \times \hat{\boldsymbol{e}}_c. \tag{1}$$

The RAC delta is propagated to the query epoch with its rate term and rotated into ECEF. With $\delta t = t - t_{\mathrm{SSR}}$ the age of the correction relative to the content epoch of the orbit record, $\boldsymbol{\delta}_{\mathrm{rac}} = (\delta_r, \delta_a, \delta_c)^{\mathrm{T}}$ its components and $\dot{\boldsymbol{\delta}}_{\mathrm{rac}}$ their rate,

$$\boxed{\boldsymbol{r}^i \;=\; \boldsymbol{r}_{\mathrm{brdc}}^i \;-\; \big[\,\hat{\boldsymbol{e}}_r \;\; \hat{\boldsymbol{e}}_a \;\; \hat{\boldsymbol{e}}_c\,\big]\big(\boldsymbol{\delta}_{\mathrm{rac}} + \dot{\boldsymbol{\delta}}_{\mathrm{rac}}\,\delta t\big).} \tag{2}$$

The subtraction follows the RTCM sign convention $\boldsymbol{\delta}$ = broadcast − precise (`products/SsrCorrectionStore.cpp`). If the broadcast velocity or the position-velocity cross product is degenerate, the frame is undefined and the broadcast position is returned unchanged. The corrected $\boldsymbol{r}^i$ enters $\rho_r^i = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ and the line-of-sight $\boldsymbol{e}_{\hat{r}}^i$ of the Foundations document unchanged. The orbit correction therefore acts on no observation equation directly, only on the geometry (the observation-geometry section) that feeds it.

The delta is applied only when its IODE matches that of the broadcast ephemeris from which the baseline orbit was evaluated (Equation (5)). Over a product orbit the gate compares with

the IODE of the orbit node nearest in time; SP3 nodes carry none, so the gate passes. On a live ephemeris cutover, the baseline position is re-referenced to the ephemeris *version* against which the SSR delta was issued, so the satellite does not drop to broadcast grade (`model/ModelGeometry.cpp`, via `LiveEphemerisStore::computeSatellitePV` with the SSR IODE). The clock is queried against the same broadcast ephemeris, so the broadcast clock's *radial* component cancels consistently against Equation (4). The decoded RAC scale factors are those of RTCM 10403.3: the radial δ_r is a 22-bit two's-complement field at 0.1 mm, along-track and cross-track are at 0.4 mm, and the rates are at 1 and 4 μm/s respectively (`rtcm/RtcmSsrDecoder.cpp`).

The orbit record also carries a *reference point* for the corrected $\boldsymbol{r}^i$. Legacy RTCM SSR corrects to the satellite antenna phase centre (APC); IGS-SSR `4076` and the network product correct to the centre of mass (CoM) (`rtcm/RtcmSsrDecoder.cpp`). Under `realtime.honor_ssr_reference_point` (default OFF, byte-identical) the geometry records whether the applied overlay was APC-referenced (`ModelGeometry::ssrOrbitApcApplied`). `PppModel` then omits its own satellite-PCO term on that geometry, so the phase-centre offset already contained in the orbit correction is not counted twice.

1.2 Clock correction: refines $-c\,\mathrm{d}t^i$

The clock message carries a quadratic polynomial in the time since the reference epoch, optionally augmented by an additive high-rate delta δC_{hr}. The total satellite-clock correction, in metres, is

$$\Delta C^i(t) \;=\; C_0 + C_1\,\delta t + C_2\,\delta t^2 + \delta C_{\mathrm{hr}}, \qquad \delta t = t - t_{\mathrm{SSR}}, \tag{3}$$

converted to seconds (`SsrCorrectionStore::applyClockToBias`) and *added* to the baseline clock (`corrections/GnssCorrections.cpp`):

$$\boxed{c\,\mathrm{d}t^i \;=\; c\,\mathrm{d}t^i_{\mathrm{base}} \;+\; \Delta C^i(t),} \tag{4}$$

where $\mathrm{d}t^i_{\mathrm{base}}$ is the precise CLK sample (post-processing) or the broadcast clock polynomial (live mode, `products/LiveEphemerisStore`). The term enters both undifferenced observation equations through $-c\,\mathrm{d}t^i$, identically on code and phase. The high-rate clock is added to the polynomial and does not replace it (`corrections/GnssCorrections.cpp`); it is subject to its own freshness test in the store. The C_2 field is a 27-bit two's-complement quantity at $2 \times 10^{-8}\,\mathrm{m/s^2}$. The time base of the polynomial is the record's content epoch (the provider's reference epoch), not its arrival time.

1.3 IOD gate at correction-set boundaries

Orbit and clock corrections are deltas against a specific broadcast ephemeris, so they are applied only when the SSR record's IODE matches that of the broadcast ephemeris from which the baseline orbit/clock was evaluated:

$$\text{apply} \iff \big(\mathrm{IODE}_{\mathrm{brdc}} < 0\big) \;\vee\; \big(\mathrm{IODE}_{\mathrm{SSR}} < 0\big) \;\vee\; \big(\mathrm{IODE}_{\mathrm{brdc}} = \mathrm{IODE}_{\mathrm{SSR}}\big), \tag{5}$$

The gate is permissive when either side is unknown (`SsrCorrectionStore::iodeMatches`, `products/SsrCorrectionStore.hpp`). A combined orbit+clock record (RTCM 1060/1066, IGS-SSR sub-type 3) takes precedence over separate messages because it guarantees a single shared IODE (`products/SsrCorrectionStore.cpp`). The two overlays gate on separate anchors: the orbit delta on the record's orbit `iode` (`model/ModelGeometry.cpp`), the clock polynomial on a distinct `clock_iode` field (`corrections/GnssCorrections.cpp`). For a standard stream, `clock_`

`iode` inherits the orbit anchor in `getOrbitClock`, so the two coincide and (5) is a single gate. Only a network clock that stamps its own clock IODE (the `brdc_rts` clock mode) makes them differ. At a correction-set boundary (the provider switches to a new ephemeris reference, hence a new IODE), the gate (5) declines the mismatched record until the matching broadcast ephemeris is available. This avoids a discontinuous jump in $\boldsymbol{r}^i$ or $c\,\mathrm{d}t^i$.

The two sides of (5) are compared in the IGS-SSR convention. A `4076` message carries an 8-bit IODE field for every constellation. A legacy Galileo message carries the full 10-bit IODnav, and the decoder keeps its low 8 bits (`normalizeSsrIode` in `rtcm/RtcmSsrDecoder.cpp`). The broadcast side converts its record in the same way (`LiveEphemerisStore::ssrIode`): the GPS and QZSS IODE as broadcast, the low 8 bits of the Galileo IODnav, and $\lfloor t_{oe}/720 \rfloor \bmod 240$ for BeiDou, with t_{oe} the seconds of week of the ephemeris reference time in BDT. The BeiDou broadcast AODE is a different quantity and is not used.

Records older than `max_age_s` (`processing.ssr_max_age_s`, default 30 s) are treated as absent. The age is that of the record's receive time relative to the query epoch, and the test is symmetric: a record dated more than the same limit ahead of the observation (a desynchronised live stream) is also rejected. The stale-hold rule extends this binary clock gate to three stages (`SsrCorrectionStore::classifyClockAge`). With `processing.ssr_clock_stale_hold_factor` $f > 1$, a clock aged in (`max_age_s`, $f \cdot$ `max_age_s`] is *StaleHeld*. It is still applied, extrapolated over the full age: the code evaluates the whole polynomial (3), of which $C_0 + C_1\,\delta t$ is the dominant part. The variance of the consuming rows grows by $q\,\delta t^3/3$, the zero-order-hold variance of a clock that wanders with density q (`network.ura_staleness_q2`, which defaults to `network.clock_model_q2`; $1.5 \times 10^{-7}\,\mathrm{m}^2/\mathrm{s}^3$ is used when it is zero; `measurement/MeasurementModelVariance.cpp`). A record beyond the hold window is *Rejected*. The default $f = 0$ reduces the rule to the plain fresh-or-absent gate (byte-identical).

Orbit-coherence echo. A network clock in the `brdc_rts` mode is estimated against one orbit update, and the radial error of the broadcast orbit is absorbed one for one into that clock. A user that applies a different orbit update receives a range bias of decimetres to metres that residual tests do not detect. The network therefore echoes the identity of the orbit it used (its IOD-SSR and content epoch) on each clock record, and the user compares it with the orbit it applies (`SsrCorrectionStore::classifyOrbitEcho`). The verdict is one of: no echo stamped, match, no applied orbit, IOD-SSR mismatch, or content-epoch skew beyond a budget. Every verdict other than a match is counted per satellite. The clock is withheld only when `processing.network_consumer.orbit_clock_max_skew_s` is positive (default 0: count only), so a stream that stamps no echo is unaffected.

1.4 Satellite biases: refine $-B_s^i$ and $-b_s^i$

SSR delivers *observable-specific biases* (OSB): one value per (satellite, RINEX-3 signal) pair, in metres. They are applied as a known additive correction to the modelled observable, with no state and no Jacobian column (the `SsrBias` term built by `addSsrBias` in `measurement/ObservationEquation.cpp`). The code OSB refines B_s^i in the undifferenced code equation:

$$\boxed{P_{r,s}^i = \rho_r^i + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t_r^{(g)} + T_r^i + \mu_s I_{r,1}^i + B_{r,s} \underbrace{- B_s^i}_{\text{SSR OSB}} + \varepsilon^P,} \tag{6}$$

and the phase OSB refines b_s^i in the undifferenced phase equation:

$$\boxed{\Phi_{r,s}^i = \rho_r^i + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t_r^{(g)} + T_r^i - \mu_s I_{r,1}^i + \lambda_s N_{r,s}^i + b_{r,s} \underbrace{- b_s^i}_{\text{SSR OSB}} + \varepsilon^\Phi.} \tag{7}$$

The stored value is the satellite bias as it is present in the observation (the underbraced term), so it is added to the clean model prediction with sign +1 (`kSatBiasSign` in `measurement/ObservationEquation.hpp`), the constant that is also the partial of the network estimator's satellite code-bias and phase-bias states. Code and phase share it. The sign relating the value a provider sends to the stored value is fixed in the bias bodies of `rtcm/RtcmSsrDecoder.cpp`. It can be confirmed only empirically against a provider's stream: float ambiguity fractions that cluster near one half instead of zero indicate an inverted constant. The value enters the predicted observable as the `SsrBias` term of the observation equation, summed with the receiver clock, troposphere, ionosphere, estimated code and phase biases and the ambiguity. Biases are queried per *raw* signal only: an ionosphere-free composite key such as `C1W_C5X` returns zero (code) or `nullopt` (phase) and is skipped, consistent with the uncombined parameterisation. The code OSB is a 14-bit field at 0.01 m, the phase OSB a 20-bit field at 0.1 mm (`rtcm/RtcmSsrDecoder.cpp`).

The 5-bit signal identifier of each bias is translated through the SSR table (`RtcmSignalMap::ssrToRinexSuffix`), which differs from the MSM table that numbers the cells of an observation message. GPS L1 C/A is SSR identifier 0 and L2 Z-tracking is 11, against MSM identifiers 2 and 10; only a few identifiers coincide (Galileo E1 C is 2 in both) (`rtcm/RtcmSignalMap.hpp`). A bias whose identifier has no entry for its constellation is dropped by the decoder, and a bias read through the wrong table would be applied to the rows of another signal.

By default the application path has no age gate: the bias is loaded whenever the store carries the exact RINEX signal key (or, for code only and with the optional band fallback `processing.ssr_bias_band_fallback` armed, a same-band sibling key; phase never falls back, to preserve the integer property). A lookup that misses the exact key while a same-band sibling exists is counted per satellite, signal and domain, whether or not the fallback resolves it. The optional staleness gate (`processing.enable_ssr_bias_max_age`, default OFF) measures the age of the bias record against the observation epoch. Its limit is `ssr_bias_stale_hold_factor` × `processing.ssr_bias_max_age_s`; the factor defaults to 2 and the maximum age defaults to the clock maximum age, hence 30 s. Below that limit the last value stays applied at full weight across staleness gaps, so a provider update interval near the maximum age produces no model step; a bias older than the limit is dropped (`measurement/ObservationEquation.cpp`). The derived configuration of the closed-loop user and the standalone `network_consumer` profile arm the gate under bias consumption unless the configuration sets the key explicitly. The consequence of the application path for the state vector, avoiding a redundant estimated bias on the same row, is the subject of the next subsection.

1.5 Satellite-bias state gate and SSR code biases

The uncombined model can also *estimate* a per-(satellite, signal) code-bias state (`estimation.estimate_sat_biases`; see the state-vector section). The estimated state and a known SSR code OSB both enter the code row with unit coefficient, so allowing both for the same (i, s) is redundant: the stream already fixes the satellite code bias, and a free state collinear with it is nearly unobservable and absorbs noise. State allocation therefore consults both correction sources before creating a sat-bias state (`MeasurementModel::ensureRequiredStates` in `measurement/MeasurementModelStates.cpp`):

$$\text{create sat-bias}(i,s) \iff \neg\,\text{hasBIA}(i,s) \;\wedge\; \neg\,\text{hasSsrCB}(i,s) \;\wedge\; \text{pivotCovers}(i,s), \tag{8}$$

where hasBIA tests the static `.BIA` product (`biaCoversSignal`, from `PppModel::hasBiasEntry`), hasSsrCB tests the live stream (`SsrCorrectionStore::hasCodeBias`, surfaced as `ssrCoversCodeSignal` in `measurement/MeasurementModelRow.cpp`) and pivotCovers is the band gate of between-satellite differencing (`signalCoveredByPivot`): only signals that the

pivot satellite tracks are observable in a single difference, and a state created for another signal would be seeded against an unconverged state vector, so its creation waits until the signal is usable. On a GLONASS FDMA signal that carries a per-slot inter-frequency-bias state, no sat-bias state is created either, because the two would be collinear. The gate rests on two design points:

- **Presence, not magnitude.** `hasCodeBias` tests whether the key is *present* in the per-signal map, which differs from `getCodeBias` $\neq 0$: a provider's present-but-zero entry still defines that signal's datum, so the estimator must not re-estimate it. The two queries agree by construction, including the optional same-band fallback, so a covered signal is never both estimated and SSR-corrected on one row. With the staleness gate armed, the presence test uses the same age limit as the application path; otherwise it ignores age.
- **Per-signal, not global.** IGS streams are partial (for example, Galileo E1 covered and E5a/E5b not). The gate keys on the exact (i, s) pair. A signal the stream does not carry keeps its estimated state, its only available code correction, while a covered signal on the same satellite is left to the OSB. A strict per-epoch prune (`pruneSatBiasStatesExcept`) drops the state when SSR coverage arrives mid-arc and re-creates it (warm-restarted from the stashed cache) if coverage later disappears.

In post-processing the live store is empty, so hasSsrCB is always false and the gate reduces to the static-`.BIA` test.

Exemption of the network estimator. When the engine is the network estimator (`ProcessingMode::PPP_RTK_Network`), the roles are reversed: it *estimates* the satellite code and phase biases as states and emits them as a product (the Network mode document). Importing SSR biases there would count them twice and make those states unobservable. Both the application path (6), (7) (`addSsrBias` adds no term) and the hasSsrCB branch of the state gate (8) (`ssrCoversCodeSignal` returns false) are therefore disabled for the network, whose own admission owns the satellite code-bias states. The network's emitter (`products/NetworkProductEmitter.cpp`, called from `network/emit/EmitStore.cpp`) pushes the estimated clocks and code and phase biases into a separate store instance, which closes the loop against a user at a test station (the closed-loop section of the Network mode document).

Integer recoverability via consistent phase biases. Removing b_s^i restores the integer property of $N_{r,s}^i$ stated in the phase observation equation of the Foundations document. With the satellite phase bias known, the remaining non-integer part of the ambiguity is the receiver phase bias $b_{r,s}$ alone, which cancels under between-satellite single differencing $\Delta\cdot$ (the differencing section). The result is an integer single-difference ambiguity admissible to AR. This defines a *consistent* phase-bias product in operational terms: the provider must generate b_s^i on the same network datum whose integer property the rover then uses. The phase-bias message therefore carries three metadata fields per signal that AR depends on (`rtcm/RtcmTypes.hpp`): an integer-preservation indicator, a two-bit wide-lane integer indicator, and a four-bit *discontinuity counter*. The message header carries two further one-bit indicators, the dispersive-bias and the Melbourne-Wübbena consistency indicators. They are read once, after the solution identifier and before the satellite count, and copied onto every record of the message (`decodeSsrHeader` in `rtcm/RtcmSsrDecoder.cpp`); they are not repeated in the satellite block. This holds for the legacy messages `1265` to `1270` and for `4076`. A decoder that reads them per satellite shifts the count and every block after the first. The satellite block also holds a yaw angle (9 bits, 180/256 degrees per unit) and a yaw rate (8 bits, 180/8192 degrees per second per unit), which are stored and not used.

1.6 Phase-bias steps and the user's ambiguities

A phase bias that changes between correction sets moves the term that the model adds to every phase row of the satellite. Let $\tilde{b}_s^i$ be the applied bias (zero when none is applied, which includes a bias dropped by the staleness gate) and $A_s^i = \lambda_s N_{r,s}^i$ the float ambiguity state in metres. The predicted phase carries $\tilde{b}_s^i + A_s^i$, both with coefficient $+1$. A change $\Delta = \tilde{b}_k - \tilde{b}_{k-1}$ between consecutive epochs carries no information about the position, and the phase model stays consistent only if the ambiguity absorbs it exactly:

$$A_s^i \;\leftarrow\; A_s^i - \Delta, \qquad \text{covariance unchanged.} \tag{9}$$

A step is signalled in two ways. An incremented *discontinuity counter* (`StateManager::noteSsrDiscontinuity`) declares that the provider's bias jumped, which breaks the integer connection of that arc. Under the staleness gate, a bias that changes between applied and dropped (a freshness flip) steps the model term without any counter. A step can also occur with no declaration. The user reacts in `RealTimeEngine::resetOnPhaseBiasDisc`, in this order.

1. A bias whose store entry carries a nonzero hold generation (the network's phase-bias hold) is claimed first by `reactToPbHold_`. Its answer, including the presence edges of a held bias, is in the user's answer in the event-response section.
2. With `processing.ssr_phase_bias_step_translate` (default OFF) every step with $|\Delta|$ above `processing.ssr_phase_bias_step_translate_eps_m` (default 5 mm) is translated by (9), declared or not (`PbStepTranslateLedger`). Inside the band the filter absorbs the move as noise and nothing is written.
3. Otherwise, with `processing.ssr_phase_bias_disc_carry` (default OFF), a declared step is translated by (9) (`PbDiscCarryLedger`). A counter change and a freshness flip on one key in one epoch are one event and one translation.
4. Otherwise the ambiguity is reset: the state is re-seeded from code at the configured ambiguity σ (`StateManager::resetAmbiguity`).

Both translations record, for every (satellite, signal), the value the model applied at the previous update, read as the model reads it (age-gated when the staleness gate is armed), so Δ is the step the phase row actually took. The declared-step carry reads an unapplied bias as zero, so a freshness flip is translated by the whole bias. The every-step translation needs an applied value on both sides and sends a declared flip to the reset. Both fall back to the reset, with each reason counted, when the key has no recorded value (a first sighting, or the first event after a restore), when the ambiguity basis is narrow-lane and wide-lane (the shift would need the basis transform), when the user is between-satellite differenced (its states are differences) or when the ambiguity key is not in the state. The every-step translation counts an undeclared step in these classes without resetting. Each event takes exactly one counted outcome, and the run-end counters close. The record of applied values is not part of the filter-state snapshot, so the first declared step per key after a warm restore takes the reset (the filter-state snapshot section).

Both settings are off in the reference configurations (`config/golden/golden_field_refound.json` and `config/golden/golden_sim_refound.json`) and carry the ledger verdict KEEP_OFF. The declared-step carry is exact and does not move the emitted product, but failed two of five bars in its re-grade (`docs/record/refound/CC-arm_pb_disc_carry_rf0hv_2026-09-15.md`). The every-step translation was superseded by the network's phase-bias hold, which changes what the product declares (`docs/record/refound/PBH_phase_bias_hold_2026-09-19.md`; the emitted phase bias). The reset is the phase-bias counterpart of the orbit and clock IOD gate (5): both protect

state continuity across correction-set boundaries, one by declining a mismatched delta, the other by resetting the dependent ambiguity. A change of the network's solution-generation id, and a re-reference of its code-bias or clock datum, are answered by resets and re-inflations described in the per-epoch update of the Real-time engine document.

1.7 GLONASS

GLONASS SSR is decoded and stored in the same way as the CDMA constellations. The legacy block (orbit 1063, clock 1064, code bias 1065, combined 1066, URA 1067, high-rate clock 1068, phase bias 1266) and the IGS-SSR `4076` subtypes in the GLONASS message-indicator band (IM = 41–47) both map onto the same orbit, clock and bias structures (`rtcm/RtcmSsrDecoder.cpp`). The orbit, clock and bias equations above apply unchanged; nothing in Equations (2), (4), (6) and (7) is constellation-specific.

The GLONASS-specific difference is the time tag of the legacy messages; the correction mathematics is shared. Legacy GLONASS SSR (`1063` to `1068` and `1266`) carries its content epoch as a 17-bit time-of-day field (DF389, seconds since the start of the GLONASS UTC(SU)+3 h day) instead of a 20-bit GPS time of week, and its satellite identifiers have 5 bits. IGS-SSR `4076` uses one layout for every constellation (6-bit satellite identifier, 8-bit IOD, 20-bit GPS-aligned time of week), so the conversion applies to the legacy messages only. The decoder converts the legacy field to the GPS time scale through `TimeUtils::glonassTodToGpsTime` (`rtcm/RtcmSsrDecoder.cpp`), resolving the day from the parser's last known GPS stream time. Without that conversion the SSR epoch would be $\{0, 0\}$; the store would then drop every GLONASS record as stale or IOD-mismatched (the `max_age_s` gate that accompanies (5)), and the satellite would fall back to broadcast grade without any error. After conversion, the per-record content epoch drives $\delta t = t - t_{\mathrm{SSR}}$ in Equations (2) and (4) as for GPS. Whether the resulting GLONASS observable then *contributes* to a between-satellite-differenced solution is a separate estimator-side option (the FDMA per-channel receiver delay does not cancel in SD); see the differencing section.

1.8 Ionosphere: $\mu_s I^i_{r,1}$ off the stream, via the slant channel

The SSR standard defines a global VTEC spherical-harmonic message (IGS-SSR IM = 201) and regional STEC products, but this engine decodes neither. The IGS-SSR classifier rejects IM = 201 as unsupported (`rtcm/RtcmSsrDecoder.cpp`), so nothing on an RTCM stream writes the slant-iono state $I^i_{r,1}$. For a user of an RTCM stream, the ionospheric term $\mu_s I^i_{r,1}$ is therefore informed only by (i) the per-satellite iono *state* estimated by the filter and (ii) the optional GIM/IONEX deterministic prior (`atmosphere/GlobalIonosphereMap`, an independent VTEC→STEC mapping unrelated to the SSR stream).

The store has a dedicated undifferenced slant-iono channel (`SsrSlantIono`, via `SsrCorrectionStore::pushSlantIono` / `getSlantIono`) that closes the atmosphere loop outside the RTCM stream. When the network estimates a spherical-harmonic VTEC field, its emit seam (`network/emit/EmitStore`) evaluates the field (the co-estimated VTEC field) at the user's pierce points and pushes a per-LOS slant delay and 1σ. The user (`RealTimeEngine::applyNetworkIonoPrior`, run after the measurement update and before AR) folds each fresh record into an ionosphere-weighted (BLUP) pseudo-observation on the matching slant-iono state (`KalmanFilter::applyPseudoObservation`), with innovation $I_i^{\mathrm{net}} - \hat{I}_i$ and $\boldsymbol{H}$ selecting the states. This is the ionosphere-weighted constraint that makes single-epoch AR feasible. The fold is gated by `network.closed_loop.consume_network_iono` (default OFF: no rows are built and the update is byte-identical). It admits a value only when the network refreshed it in the current epoch (`max_`

age $= \frac{1}{2}$ the epoch spacing, or 30 s when the spacing is unknown), so a satellite the network did not re-supply floats instead of inheriting a stale delay.

The reference configurations estimate no spherical-harmonic field (the slant ionosphere is free per line of sight), so the supply is gated on a model that is never built and the channel stays empty, although the reference configurations carry the setting as true by lineage. The ledger verdict of the setting is KEEP_OFF (`docs/CONFIG_promotion_ledger.json`; `docs/refound/AB1_sh_consumer_product_2026-09-20.md`). The channel has no record in the product file either, so the offline replay and the file-tail user cannot receive it; the setting is inert there and draws a warning.

Three properties of the channel follow from the field and are not chosen by the user. The supply covers GPS and Galileo only: GLONASS is excluded from the SH field, so a user's GLONASS slant states float. When a second shell is armed, the pushed value is the *sum* over shells, each evaluated at its own pierce geometry, so the user does not depend on the layer model. The per-LOS σ is the field's representativeness floor (the SH representativeness sigma). Under `network.iono_supply_formal_sigma` (default OFF, byte-identical) it is root-sum-squared with the SH coefficients' *formal* σ, propagated through the same weighted basis, and the supply is skipped for a satellite that has no estimated satellite code bias (OSB) in the current network solution.

At the user the rows are not independent. They are folded as a per-constellation common-mode block with the noise structure of the GIM iono-weighting covariance: rows of one constellation share the variance `common_mode_sigma_m`^{2} in addition to their own diagonal variance. Frozen BSSD pivots, and any slant state with zero variance, are excluded for the reason given there: the common-mode block absorbs the datum offset of the *remaining* rows, so the datum satellite must not appear as a measurement.

A standardised STEC product on the stream remains open: the in-process channel is a closed-loop store hand-off, not an IGS-SSR or RTCM message that a user of a third-party stream can decode.

1.9 The user of the network's corrections

The network's products reach a user engine in three ways: the in-process closed loop, in which the network's emit seam fills an owned store each epoch; the offline replay of a recorded product file (`network.closed_loop.replay_products_path`); and, in a real-time run, a tail of a growing product file (`realtime.network_products_path`). The first two are described in the closed-loop section of the Network mode document. Each gives the engine an owned `SsrCorrectionStore` (`RealTimeEngine::setSsrStore`) in place of the process-wide one.

The configuration of the user is derived from the configuration of the network by `GnssConfig::deriveClosedLoopConsumerConfig`. The standalone profile `processing.network_consumer` applies the same couplings (`applyNetworkConsumerCouplings`). The derived configuration differs from that of the network in these respects.

- The processing mode is the single-receiver PPP-RTK user. Estimation of satellite clocks, satellite phase biases, satellite code biases and the network ionosphere is switched off, because the corrections supply them.
- Residual satellite code biases are estimated only when the user does not take the network clock (`estimate_sat_biases` is the negation of clock consumption). When no satellite-bias state is estimated, the variance of a code row on a signal that neither a `.BIA` entry nor the stream covers is multiplied by the ordinary user factor 100 (`no_bia_code_downweight`), in place of the weight the network gives it.

- The undifferenced user applies no observation differencing. The phase biases are applied to the raw float (7), and AR forms between-satellite differences at fix time, which cancels the receiver phase bias. AR is direct per-band MLAMBDA without the Melbourne-Wübbena wide lane (`ar_wl_only` false, after which the driver sets `ar_ignore_mw_wl`), and starts only once the position σ is below 0.2 m. The reference configurations run this user float (`network.closed_loop.ar_mode` 0).
- The bias staleness gate is armed under consumption unless the configuration sets the key explicitly.
- Settings that act on the network's own solution (the canonical-signal rule, the extra-band code weight, the second code and phase channels) are pinned to neutral values, so that a case moves the network and leaves its user unchanged.

The user's answers to the network's declarations are the phase-bias reactions of Section 1.6 and the datum-break reactions of the per-epoch update of the Real-time engine document.

1.10 Provenance and code pointers

Both transports map onto the same internal correction structures (`rtcm/RtcmTypes.hpp`) through a single decoder (`rtcm/RtcmSsrDecoder.cpp`). Legacy RTCM uses per-constellation six-message blocks (orbit, clock, code bias, combined orbit+clock, URA, high-rate clock) and a separate phase-bias block (1265–1270). IGS-SSR `4076` repackages the same payloads under a message indicator (IM). Its *tens* place ($\lfloor \mathrm{IM}/10 \rfloor$) selects the constellation (GPS $2x$, GLONASS $4x$, Galileo $6x$, QZSS $8x$, BeiDou $10x$, SBAS $12x$), and its *ones* place (IM mod 10) selects the subtype (1=orbit, 2=clock, 3=combined, 4=high-rate, 5=code bias, 6=phase bias, 7=URA). This subtype ordering differs from the legacy SSR block ordering, so the legacy mapping is not reused (`RtcmSsrDecoder.cpp`). A `4076` message uses one set of field widths for every constellation (6-bit satellite identifier, 8-bit IOD, 20-bit GPS-aligned time of week), where the legacy blocks use per-constellation widths. Applying the legacy Galileo or BeiDou width to a `4076` message would over-read the IOD by two bits and shift every later field. RTCM counts SBAS satellites from 1 at PRN 120, and the decoders name identifier n as $\mathrm{S}(n+19)$, so PRN 120 is S20 as in RINEX and SBF (`rtcm/RtcmSatId.hpp`). VTEC (IM 201) and NavIC (IM 141–147) are rejected by the message-indicator classifier before the header is parsed, so they return `ok = false` with an empty header. For this reason the ionospheric term receives no SSR contribution from the stream (the `SsrSlantIono` store channel of the ionosphere subsection is unaffected).

The correction definitions Equations (2), (4) and (5) are implemented in the static, deterministic `applyOrbitToPosition` / `applyClockToBias` / `iodeMatches` members of `products/SsrCorrectionStore`, the interface this document describes. The stateful streaming layer (the mutex-guarded per-satellite cache with freshness gating on receive time, the `RealTimeEngine::feedRtcmFrame` push loop, and the obs↔SSR synchronisation that bounds how far the obs stream may run ahead of lagging corrections) is described in the Real-time engine document. It is kept separate so that the correction mathematics can be analysed and tested in isolation. The estimator-side warm-up gate `estimation.defer_until_corrections_ready.enabled` (`src/core/config/GnssConfigLoad.cpp`; default OFF, real-time only) defers estimation until a quorum of satellites is centimetre-grade from broadcast plus SSR (by default at least 5 in total and 4 per constellation). The filter then seeds code and phase together, instead of cold-starting on metre-grade broadcast clocks and fixing a contaminated ionosphere/bias/ambiguity partition.

The store also carries a per-epoch *heartbeat* of the network (`NetworkProducerStatus`). It lets a user distinguish "the network withheld this epoch" from "the network stopped": a withheld epoch still writes its status line, although it publishes no corrections. The token set of the product file is forward-compatible: an unrecognised token from a newer network maps to `State::Unknown` and is

reported, never fatal (`NetworkProducerStatus::parseState` / `stateName`, declared in the store's header). The token `withheld_sh` is retained so that archived product files parse; the spherical-harmonic health gate that wrote it was deleted from the network, and no current product contains it.

References

[1] International GNSS Service Real-Time Working Group. *IGS State Space Representation (SSR) Format, Version 1.00*. Format specification. RTCM message 4076 IGS-SSR sub-types. International GNSS Service (IGS), 2020.

[2] *RTCM Standard 10403.3: Differential GNSS Services – Version 3*. Special Committee No. 104; State Space Representation messages 1057–1068, 1240–1270, 4076. Radio Technical Commission for Maritime Services. Arlington, VA, USA, 2016.